



University
of Exeter

COM3031 Probabilistic Machine Learning: Week 8

Term 2, 2025-2026

Dr Zeyu Fu, Lecturer in Computer Vision
and Machine Learning

z.fu@exeter.ac.uk

In this week

Lectures:

- Sequential Data and Markov Chain
- Hidden Markov Models (HMM)
- Three Fundamental Problems in HMM
 - Likelihood → Forward Algorithm
 - Decoding → Viterbi Algorithm
 - Learning → Baum–Welch Algorithm

Workshop:

- Variational Autoencoders

Part I: Sequential Data and Markov Chain

From Latent Variables to Temporal Structure

What we have learned so far

- Latent variable models:

$$p(x) = \int p(x|z)p(z)dz$$

- Hidden variable z explains observation x
- Encoder approximates $q(z | x)$
- Decoder models $p(x | z)$
- Learning = reduce uncertainty about hidden structure
- Representation learning = discover latent causes
- Generative models = explain how data is produced

Limitation

- One observation at a time and no notion of temporal dependence

New Question

- What if: $x_1, x_2, \dots, x_T$ are *not independent*?

When Data Is Sequential

When Observations Are Not Independent

- Independent data assumption (previous weeks),

$$x^{(1)}, x^{(2)}, \dots, x^{(N)} \sim \text{i.i.d.}$$

- Likelihood:

$$p(x^{(1)}, \dots, x^{(N)}) = \prod_i p(x^{(i)})$$

But in many problems:

- Observations form a sequence $x_1, x_2, \dots, x_T$, and typically: x_t depends on x_{t-1}

Examples

- Speech: phoneme depends on previous phoneme
- Text: next word depends on previous words
- Finance: market regime persists over time
- Weather: today's weather influences tomorrow's

New challenge: How do we model dependence over time?

Sequential Data: Key Properties University of Exeter

Temporal Dependence

- Observations are ordered:

$$x_1, x_2, \dots, x_T$$

- The present depends on the past:

$$x_t \text{ depends on } x_{t-1}$$

- Example: the next word depends on the previous word.

Latent/Hidden State Dynamics

- The system evolves over time.
- There may be an underlying **state**:
 z_t influences z_{t+1}
- We observe signals, not the true state.

Stationarity Assumption (Often Made)

- The transition mechanism does not change over time:
- $p(z_t | z_{t-1})$ is time-invariant
- Example: grammar rules in language stay the same.

Prediction Goal

- Use past information to infer hidden state
- Predict future observations:
 $p(x_{t+1} | x_{1:t})$

Markov Assumption

Example sequence:

- Walk $\rightarrow$ Shop $\rightarrow$ Clean o_1, o_2, o_3
- Full chain rule (no assumptions):

$$p(o_1, \dots, o_T) = \prod_{i=1}^T p(o_i \mid o_1, \dots, o_{i-1})$$

- Captures full dependence
- Too many conditional probabilities
- Intractable in practice

Key Problem: we need a model that:

- Captures dependence
- Remains computationally manageable

First-Order Markov Assumption

$$p(o_t \mid o_1, \dots, o_{t-1}) = p(o_t \mid o_{t-1})$$

- The future depends only on the present.

Intuition:

- Weather tomorrow depends on today, not directly on last week.
- The next word depends mostly on the previous word.
- Market regime today depends on yesterday's regime.

Markov Chain

Joint distribution under Markov assumption:

$$p(o_1, \dots, o_T) = p(o_1) \prod_{i=2}^T p(o_i | o_{i-1})$$

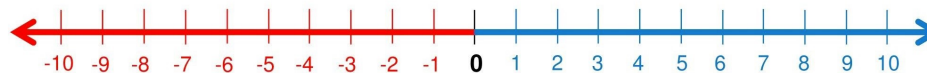
Homogeneous (Time-Invariant) Markov Chain

If transition probabilities do not change over time, then the chain is homogeneous:

$$p(o_t | o_{t-1}) = p(o_{t+1} | o_t)$$

- Transition rules stay constant
- Simplifies modeling

Example: Random (Drunkard's) Walk:



At each step:

- Move +1 with probability 0.5
- Move -1 with probability 0.5

Part II: Hidden Markov Models (HMM)

Hidden Markov Models

From Markov Chains to Hidden States

- Previously: Markov chain $\rightarrow$ all states observable and we compute probabilities of sequences directly
- Now: Some states are **hidden (unobservable)** and we only observe outcomes generated from them

Example: Weather & Activities

- Hidden states:

$$s_t \in \{\text{Rainy}, \text{Sunny}\}$$

- Observed events:

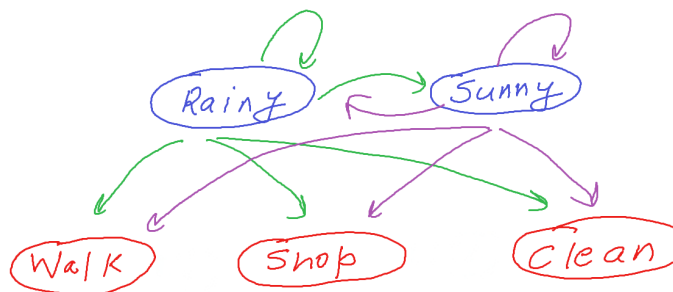
$$o_t \in \{\text{Walk}, \text{Shop}, \text{Clean}\}$$

- We observe:

$$o_1, o_2, \dots, o_T$$

- We do **not** observe:

$$s_1, s_2, \dots, s_T$$



States (Hidden)

Observations

Hidden Markov Models

- Goal: Given states S and observations O , an HMM models the **joint distribution**:

$$p(S, O)$$

Core Structure

- Hidden state dynamics (Markov property):

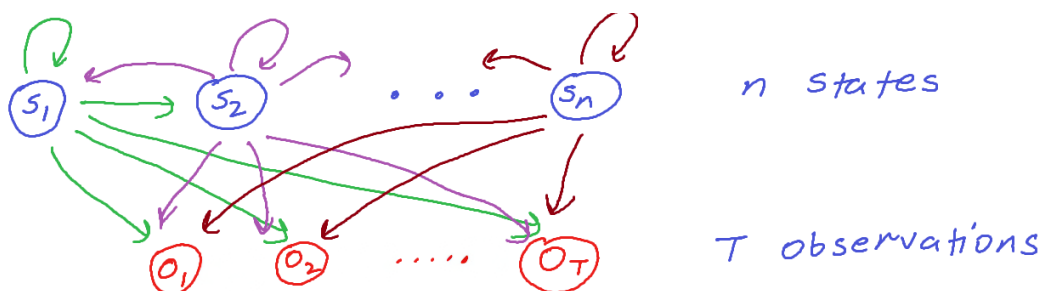
$$s_t \sim p(s_t | s_{1:t-1}) = p(s_t | s_{t-1})$$

- Hidden states evolve over time.
- Each state depends only on the previous state.

- Observation model (Emission process):

$$o_t \sim p(o_t | s_1, \dots, s_T, o_1, \dots, o_{t-1}) = p(o_t | s_t)$$

- Observations are generated from the current hidden state.



Hidden Markov Models

Key Assumptions

- Hidden states are mutually exclusive (one active at each time).
- Hidden states are dependent across time.
- Observations are conditionally independent given the state.

Applications of HMM:

- Speech recognition
- Natural language processing
- Handwriting recognition
- Biological sequence modelling (DNA, proteins)

Model Notation:

- States = $s_1, s_2, \dots, s_n$
- Observation = $o_1, o_2, \dots, o_T$
- A is transition probability matrix
- B is emission probability matrix
- π is transition probability vector
- A hidden Markov model can be represented as $\lambda = (A, B, \pi)$

Key Notation in HMMs

Hidden States

Let the hidden states be:

$$S = \{ \text{Rainy}, \text{Sunny} \}$$

- At time t , the hidden state is:

$$s_t \in S$$

Transition Probabilities

- The transition probability:

$$a_{ij} = P(s_t = j \mid s_{t-1} = i)$$

- This means: probability of moving from state i to state j .

Transition Matrix

$$A = \begin{matrix} & \begin{matrix} \text{Rainy} & \text{Sunny} \end{matrix} \\ \begin{matrix} \text{Rainy} \\ \text{Sunny} \end{matrix} & \begin{bmatrix} 0.7 & 0.3 \\ 0.4 & 0.6 \end{bmatrix} \end{matrix}$$

- Rows = previous state
- Columns = next state
- $P(\text{Rainy} \rightarrow \text{Rainy}) = 0.7$
- $P(\text{Rainy} \rightarrow \text{Sunny}) = 0.3$
- $P(\text{Sunny} \rightarrow \text{Rainy}) = 0.4$
- $P(\text{Sunny} \rightarrow \text{Sunny}) = 0.6$

Important Property

For each state i :

$$\sum_j a_{ij} = 1$$

- Each row sums to 1 because it represents a probability distribution.

Emissions & Initial Distribution

Emission Probabilities

- The emission probability:

$$b_{ij} = P(o_t = j \mid s_t = i)$$

- probability of observing activity j given hidden state i .

Emission Matrix B

- States: $S = \{\text{Rainy}, \text{Sunny}\}$
- Observations: $O = \{\text{Walk}, \text{Shop}, \text{Clean}\}$

$$B = \begin{matrix} & \begin{matrix} \text{Walk} & \text{Shop} & \text{Clean} \end{matrix} \\ \begin{matrix} \text{Rainy} \\ \text{Sunny} \end{matrix} & \begin{bmatrix} 0.1 & 0.4 & 0.5 \\ 0.6 & 0.3 & 0.1 \end{bmatrix} \end{matrix}$$

- Rows = hidden state
- Columns = observation
- Each row sums to 1.

Initial State Distribution

$$\pi_i = P(s_1 = i)$$

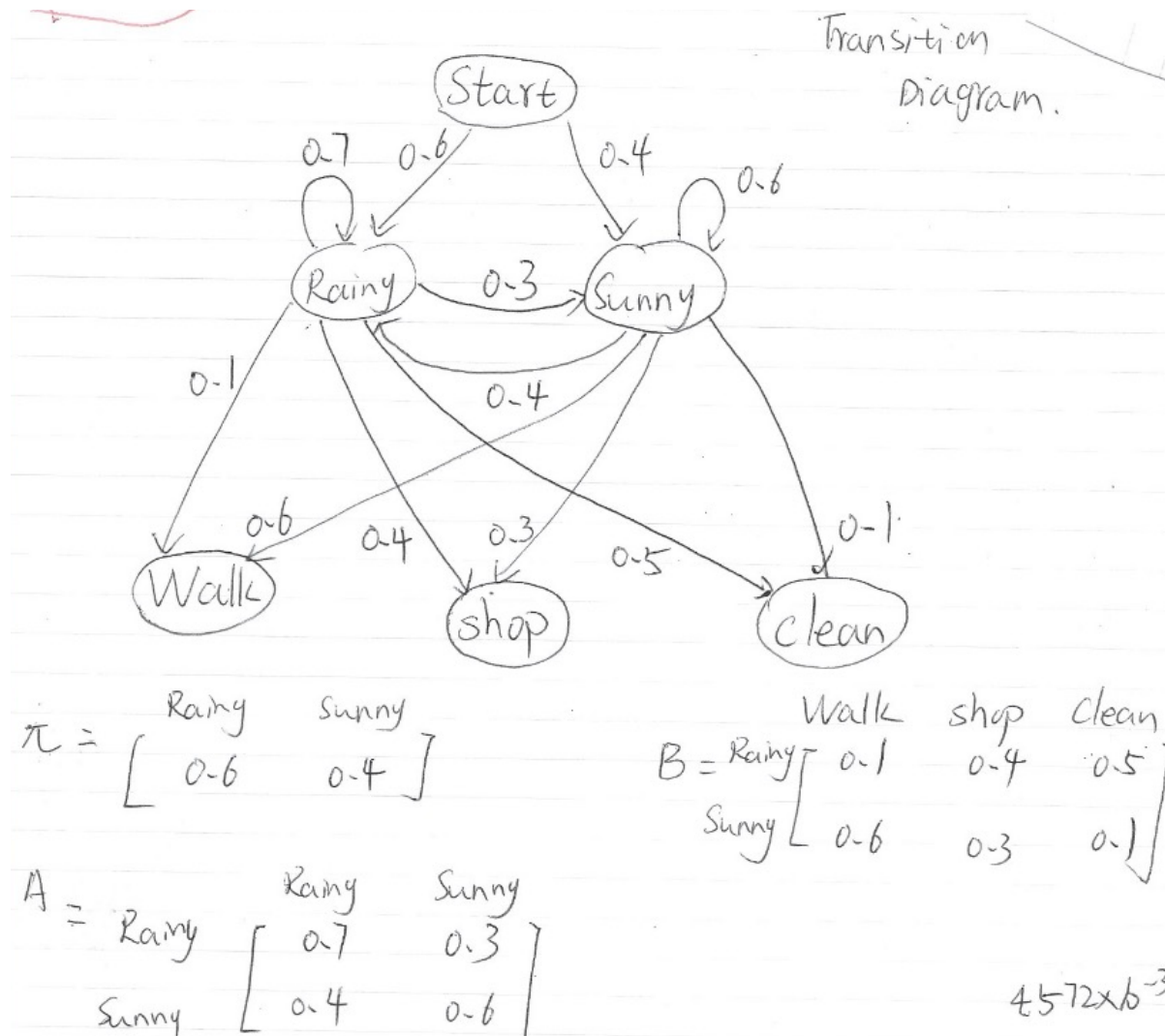
- Example:

$$\pi = \begin{matrix} & \begin{matrix} \text{Rainy} & \text{Sunny} \end{matrix} \\ \begin{bmatrix} 0.6 & 0.4 \end{bmatrix} \end{matrix}$$

- 60% chance the first day is Rainy
- 40% chance the first day is Sunny
- And:

$$\sum_i \pi_i = 1$$

Transition Diagram



From Model to Problems

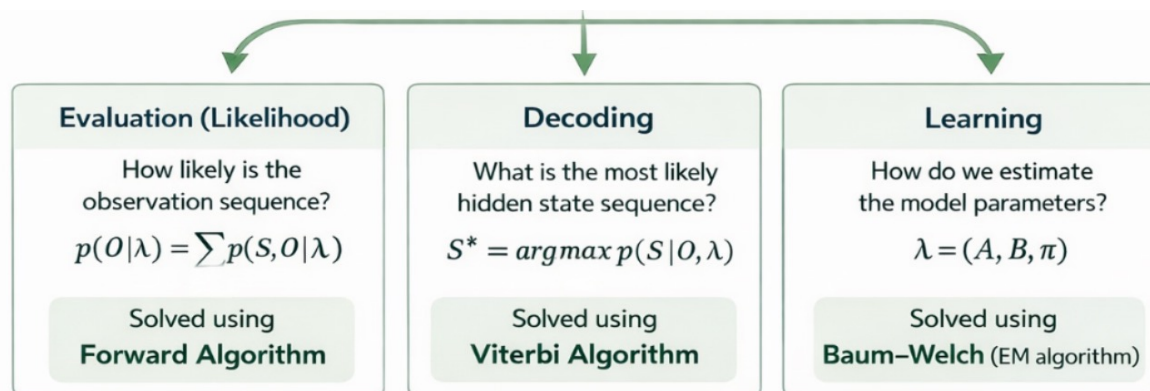
Full Joint Distribution

- For hidden states $s_{1:T}$ and observations $o_{1:T}$:
- $p(s_{1:T}, o_{1:T}) = \pi_{s_1} \prod_{t=2}^T p(s_t | s_{t-1}) \prod_{t=1}^T p(o_t | s_t)$

This decomposes into three components:

- Initial state probability : π_{s_1} ;
- State transition probabilities: $a_{ij} = p(s_t = j | s_{t-1} = i)$
- Emission probabilities: $b_j(o_t) = p(o_t | s_t = j)$

From this model, three fundamental questions arise:



Part III: Three Fundamental Problems in HMM

Evaluation Problem (Likelihood)



University
of Exeter

Goal 1: Given an HMM $\lambda = (A, B, \pi)$ and observation sequence $O = (o_1, \dots, o_T)$,

- Compute: $p(O | \lambda)$

Direct (Naive) Computation

- Since states are hidden:

$$p(O | \lambda) = \sum_S p(O, S | \lambda) = \sum_S p(O | S, \lambda) p(S | \lambda)$$

- We must sum over all possible hidden state sequences.

Example (Weather Model)

- Observation sequence: $O = (\text{Walk}, \text{Shop}, \text{Clean})$
- Hidden state sequence: $S = \{\text{Rainy}, \text{Sunny}\}$

- Possible hidden sequences:

- Total sequences: $N^T = 2^3 = 8$

Rainy	Rainy	Rainy
Rainy	Rainy	Sunny
Rainy	Sunny	Rainy
Rainy	Sunny	Sunny
Sunny	Sunny	Sunny
Sunny	Sunny	Rainy
Sunny	Rainy	Sunny
Sunny	Rainy	Rainy

Evaluation Problem (Likelihood)



University
of Exeter

Likelihood of a Single State Sequence

- For a specific hidden sequence S :

$$p(O, S|\lambda) = p(O|S, \lambda)p(S|\lambda)$$

Example (Weather Model)

- Observation sequence: $O = (\text{Walk}, \text{Shop}, \text{Clean})$
- Hidden state sequence: $S = (\text{Rainy}, \text{Rainy}, \text{Rainy})$

Emission probability:

$$p(O|S, \lambda) = p(\text{Walk}|R)p(\text{Shop}|R)p(\text{Clean}|R) = 0.1 \times 0.4 \times 0.5 = 0.02$$

State transition probability:

$$p(O, S|\lambda) = p(O|S, \lambda)p(S|\lambda) = 0.02 \times 0.294 = 0.00588$$

Final Likelihood:

- To obtain the final likelihood we sum over all possible state sequences

$$p(O|\lambda) = \sum_S p(O, S|\lambda)$$

Why Forward Algorithm

Computation cost of the naïve approach:

- For N hidden states and T observations, there are N^T possible sequences
- Direct method involves $2T \times N^T$ calculations: $(2T - 1)N^T$ multiplications and $N^T - 1$ additions.
- For $N = 5$ and $T = 100$, there are $2 \times 100 \times 5^{100} \approx 10^{72}$ calculations

Key Idea: Dynamic Programming

- The **Forward Algorithm** avoids enumerating all paths.
- Instead, it stores intermediate probabilities and reuses them.
- Define the forward variable:

$$\alpha_t(i) = p(o_1, \dots, o_t, s_t = i \mid \lambda)$$

- The probability of observing the first t observations and ending in state i .
- Computational complexity: $O(N^2T)$

Forward Algorithm

Recursive Computation

- Initialisation:

$$\alpha_1(j) = \pi_j b_j(o_1)$$

- Recursion:

$$\alpha_t(j) = \sum_{i=1}^N \alpha_{t-1}(i) a_{ij} b_j(o_t)$$

Previous forward path probability

State observation likelihood

Transition probability

- Termination

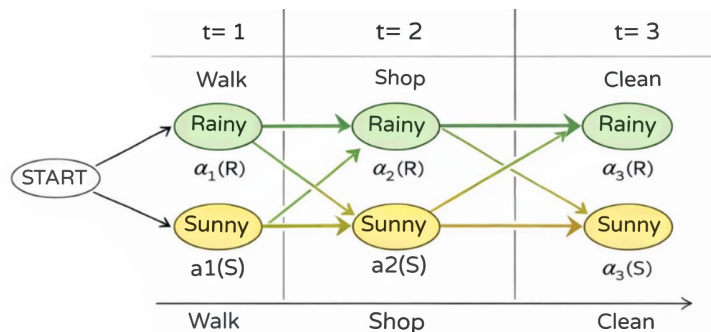
$$p(O|\lambda) = \sum_{i=1}^N \alpha_T(i)$$

Forward Algorithm

Example setup

Hidden states: Rainy and Sunny and the observation sequence: Walk-Shop-Clean

Forward Trellis:



- Each node represents the probability of reaching that state after seeing observations up to time t .
- Detailed demonstration (on board)

Forward algorithm avoids enumerating all paths, reuses intermediate probabilities and computes efficiently.

Decoding Problem

Goal 2: Given an HMM $\lambda = (A, B, \pi)$ and an observation sequence

$$O = (o_1, o_2, \dots, o_T)$$

- Find the **most likely hidden state sequence**

$$S^* = \arg \max_S P(S | O, \lambda) = \arg \max_S P(S, O | \lambda)$$

Naive Solution

- Enumerate all possible hidden state sequences
- Compute the joint probability: $P(S, O | \lambda)$
- For each sequence, choose the sequence with the maximum probability

Why This Is Infeasible

- For N hidden states and T observations, there are N^T possible state sequences.
- Example: $N = 2, T = 3, 2^3 = 8$ possible sequences
- For real problems, this grows exponentially.

Viterbi Algorithm

Key Idea:

- Instead of evaluating every possible hidden state sequence, we use **dynamic programming** to efficiently compute the most likely path.

Viterbi Variables:

- Define the Viterbi score

$$v_t(j) = \max_{s_1, \dots, s_{t-1}} P(s_1, \dots, s_{t-1}, s_t = j, o_1, \dots, o_t \mid \lambda)$$

- This represents: the probability of the best path ending in state j at time t .

Backpointer Variable:

$$\psi_t(j) = \arg \max_i [v_{t-1}(i) a_{ij}]$$

- Stores which previous state gives the best path.

Key insights:

- Finds the most likely hidden state sequence
- Avoids enumerating all N^T paths

Viterbi Algorithm

Recursive computation:

- Initialisation:

$$v_1(j) = \pi_j b_j(o_1)$$

- Recursion ($t = 2 \dots T$):

$$v_t(j) = \max_{i=1}^N [v_{t-1}(i) a_{ij}] b_j(o_t)$$

Previous Viterbi path
probability

Store the best previous state:

$$\psi_t(j) = \arg \max_i [v_{t-1}(i), a_{ij}]$$

- Termination

$$s_T^* = \arg \max_j v_T(j)$$

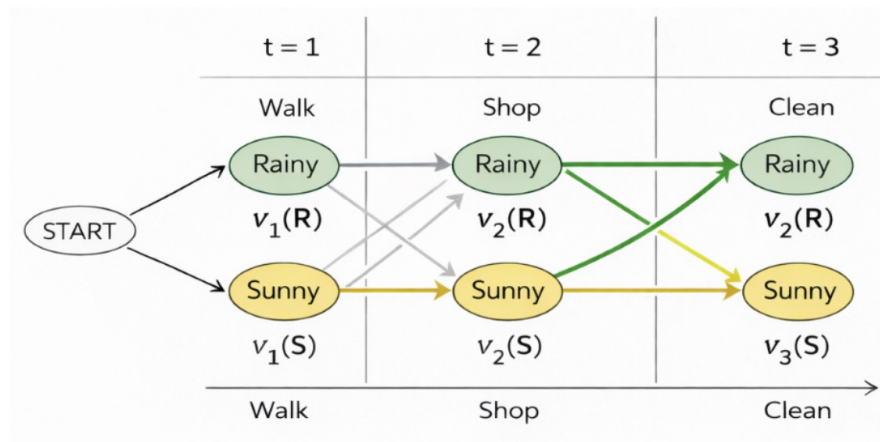
Backtrack using $\psi_t(j)$ to recover the optimal state sequence.

Viterbi Algorithm

Example setup

Hidden states: Rainy and Sunny and the observation sequence: Walk-Shop-Clean

Viterbi Trellis:



- Each node stores the probability of the best path reaching that state.
- At each step we check all incoming transitions, keep only the maximum probability path and store the previous state using $\psi_t(j)$.
- Backtracking gives the most likely sequence: *Sunny* $\rightarrow$ *Rainy* $\rightarrow$ *Rainy*
- Detailed demonstration (on board)

Learning Problem

Goal 3: Given **only an observation sequence**,

$$O = (o_1, o_2, \dots, o_T)$$

- Estimate the HMM parameters $\lambda = (A, B, \pi)$ that maximize the likelihood $P(O | \lambda)$

Unknown Parameters

- Transition matrix: $A = \{a_{ij}\}$, probability of transitioning from state i to state j
- Emission matrix: $B = \{b_j(o)\}$, probability that state j generates observation o
- Initial distribution: $\pi_i = P(s_1 = i)$, probability that the sequence starts in state i

Challenge

- The hidden states S are not observed
- We must estimate parameters while the states are unknown

Key idea

- We estimate parameters by inferring hidden states probabilistically.
- This leads to the **Baum–Welch algorithm**.

Baum–Welch Algorithm

- Baum–Welch is also called the **Forward–Backward** algorithm
- It is a special case of the **Expectation–Maximisation (EM)** algorithm.

Iterative Learning Procedure

Step 1 — Initialization

- Start with **initial guesses** A, B, π (often random or uniform)

Step 2 — Expectation Step (E-step)

- Use **Forward–Backward** to compute: probability of being in state i at time t

$$\gamma_t(i) = P(s_t = i \mid O, \lambda)$$

- And compute probability of transitioning from i to j

$$\xi_t(i, j) = P(s_t = i, s_{t+1} = j \mid O, \lambda)$$

Step 3 — Maximisation Step (M-step)

- Update parameters using these expected counts:
- Transition probabilities

$$a_{ij} = \frac{\sum_{t=1}^{T-1} \xi_t(i, j)}{\sum_{t=1}^{T-1} \gamma_t(i)}$$

Baum–Welch Algorithm

Step 3 — Maximisation Step (M-step)

- Update emission probabilities

$$b_j(o_k) = \frac{\sum_{t:o_t=o_k} \gamma_t(j)}{\sum_{t=1}^T \gamma_t(j)}$$

- Initial probabilities

$$\pi_i = \gamma_1(i)$$

Step 4 — Repeat

- Repeat **E-step and M-step** until convergence.
- The likelihood $P(O|\lambda)$ increases at each iteration.

Baum–Welch Algorithm

- estimates how often each state occurs
- estimates how often transitions occur
- updates parameters based on those expectations

Summary

Key Idea:

- **HMM** connects **observed sequences** with **hidden state dynamics**
- Hidden states evolve over time and follow a **Markov chain**
- Observations are generated from those states

$$s_t \sim p(s_t | s_{t-1}), \quad o_t \sim p(o_t | s_t)$$

- **Examples:** speech recognition Part-of-speech (POS) tagging, bio-sequences, activity recognition.

Three HMM problems

Given an HMM $\lambda = (A, B, \pi)$ and an observation sequence

$$O = (o_1, o_2, \dots, o_T)$$

- **Likelihood** (evaluation): $P(O | \lambda) \rightarrow$ **Forward**
- **Decoding** best hidden path $\operatorname{argmax} P(S | O, \lambda) \rightarrow$ **Viterbi**
- **Learn** parameters $\lambda = (A, B, \pi) \rightarrow$ **Baum–Welch**
- All algorithms rely on **dynamic programming**